

Lecture 18: Transfer Learning and CNNs in Practice

Standing on ImageNet's shoulders

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The 2,000-image reality

The sorting line finally needs its own model:
grade each fruit as **export**, **market**, or **juice**.
You spent a week collecting and labeling **2,000**
photos.

Predict: you train the L17 ResNet-18 on them,
from scratch. What happens?



2,000 of these - and that is all you get.

The 2,000-image reality

The sorting line finally needs its own model: grade each fruit as **export**, **market**, or **juice**. You spent a week collecting and labeling **2,000 photos**.

Predict: you train the L17 ResNet-18 on them, from scratch. What happens?

It **memorizes** the training set and falls apart on new fruit: **11.7 million** parameters vs **2,000** examples - the overfitting story ([06]) at its bluntest.

The fix is not a smaller net. It is to **not start from scratch**.



2,000 of these - and that is all you get.

Outline

Transfer learning

Seeing what the net sees

The economics of small data

Reuse features, not just architectures

Somebody already trained a net on 1.3 million images. Most of what it learned is about *seeing*, not about their classes.

Why transfer works

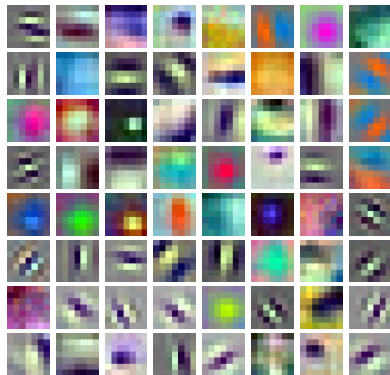
L16: conv layers learn a **feature hierarchy** - edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects.

L17: a pretrained net's conv1 filters *are* the kernel zoo - edge and color detectors nobody designed.

Edges and textures are the **same** for pomegranates, pets, and X-rays. Only the late, task-specific layers need *your* data.

A pretrained trunk is a general-purpose **visual feature extractor**. The hierarchy is the reusable asset.

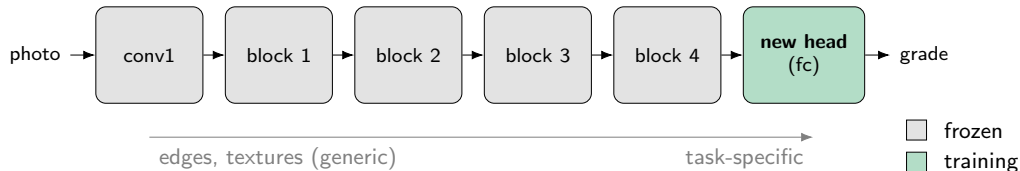
ResNet-18 first-layer filters, learned on ImageNet (64 x 7x7 RGB)



The L17 payoff figure: learned on ImageNet, useful everywhere.

Two recipes

(Click to unfreeze.)

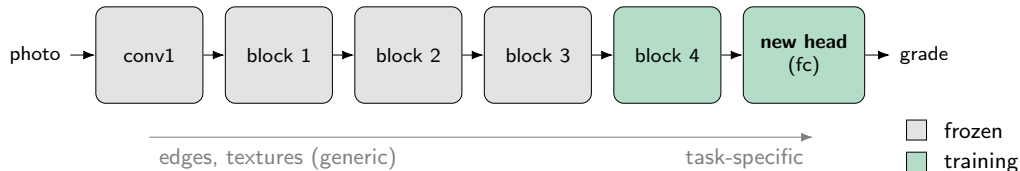


Recipe 1 - feature extraction. Freeze the whole trunk; train only a new head. The trunk is a fixed feature extractor - tiny data suffices.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

Two recipes

(Click to unfreeze.)

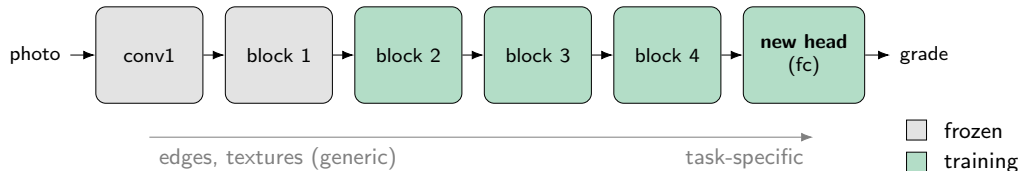


Recipe 2 - fine-tuning. Start by unfreezing the *top* block - the most task-specific features - with a **small** learning rate.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

Two recipes

(Click to unfreeze.)

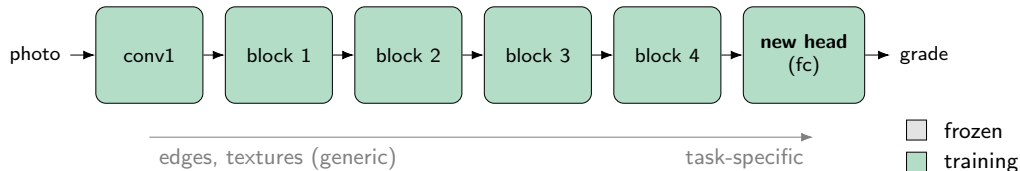


With more data, unfreeze deeper. Early layers stay frozen longest - their features are the most generic.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk - needs more data, buys a better ceiling.

Two recipes

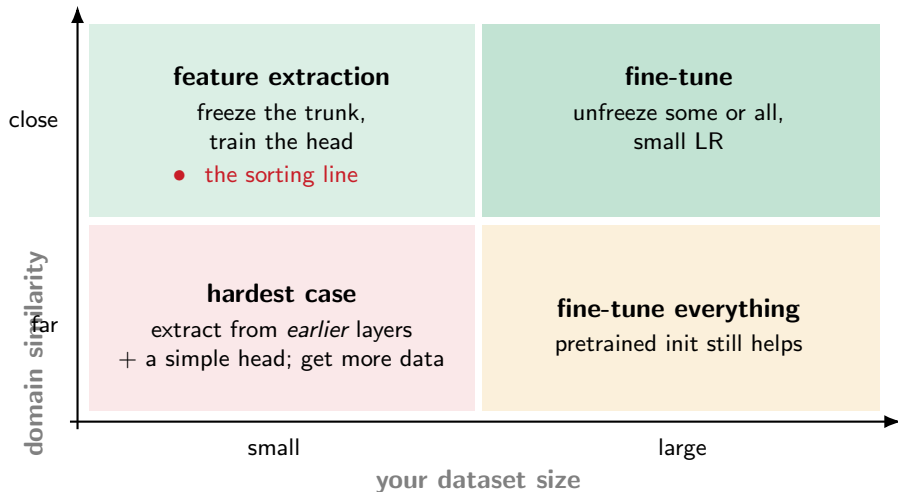
(Click to unfreeze.)



With enough data, everything trains - still at a small LR. Better ceiling, more ways to overfit.

Feature extraction = new head only, tiny data suffices. **Fine-tuning** = also nudge the trunk
- needs more data, buys a better ceiling.

Which recipe? Two questions decide



Fine-tuning in practice: you know all of this

- ▶ **Small learning rate.** You start *near a good minimum* - a big step jumps away from everything ImageNet taught the net ([44]'s learning-rate intuition).
- ▶ **Augmentation on.** Flips, crops, color jitter stretch 2,000 photos further ([43]).
- ▶ **Early stopping** on a validation split - the overfitting guardrail ([06], [43]).
- ▶ **Freeze first, unfreeze gradually** - the only genuinely new move today.

Fine-tuning is the [42]-[45] toolkit applied near a good minimum. Nothing new to learn - just new discipline.

The BatchNorm gotcha

You froze the trunk: `requires_grad = False` on every weight. Accuracy is still bad. Why?

- ▶ In `train()` mode BatchNorm normalizes each batch with **that batch's** mean and variance, and keeps updating **running averages** of them; `eval()` uses the running averages instead ([44] owns the formula).
- ▶ Those running averages are **not weights**: `requires_grad = False` does not touch them, so they keep changing while "everything is frozen".
- ▶ Small batches from *your* small dataset make the per-batch statistics noisy and drag the running averages away from the ImageNet values the frozen weights expect - accuracy drops **silently**.

How fast? PyTorch updates $\text{running} \leftarrow 0.9 \cdot \text{running} + 0.1 \cdot \text{batch}$. Measured: a frozen BN layer in `train()` mode, stored mean 0, fed 10 batches of mean 5 $\rightarrow$ running mean 3.26 - already 65% of the way $(1 - 0.9^{10})$ to your data.

Practical rule: when the trunk is frozen, put its BN layers in **eval mode**. And whenever a "frozen" model underperforms - suspect the BN statistics first.

The canonical snippet

The whole idea, in eight lines:

```
from torchvision import models
import torch, torch.nn as nn

model = models.resnet18(weights="IMAGENET1K_V1")  # 11.7M trained weights
for p in model.parameters():
    p.requires_grad = False                        # freeze the trunk
model.fc = nn.Linear(model.fc.in_features, 3)      # new head: 3 grades
optimizer = torch.optim.SGD(model.fc.parameters(),
                             lr=0.001, momentum=0.9)
# then: the usual training loop, unchanged - only the head learns
```

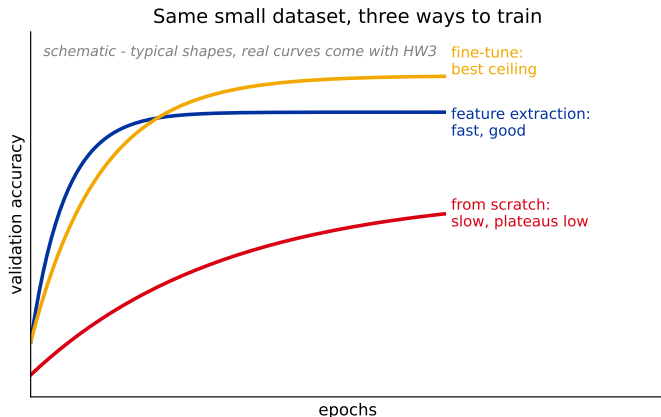
The new fc is created *after* the freeze, so it trains by default - and it is the only thing the optimizer sees.

What transfer buys you

Same small dataset, three ways to train:

- ▶ **from scratch:** slow, plateaus low - 11.7M parameters memorize 2,000 photos;
- ▶ **feature extraction:** fast and good;
- ▶ **fine-tune:** the best ceiling.

These are the *typical shapes*. You will measure the real curves yourself on Oxford-IIIT Pets in HW3.



When transfer fails

- ▶ **Domain too far.** Spectrograms, medical volumes, radar - ImageNet features stop being the right vocabulary. Expect to unfreeze more, or to rethink.
- ▶ **Bias mismatch.** ImageNet is object-centric daylight photography; if your data is not, the pretrained assumptions can work against you.

Classic silent bug: normalizing inputs with *your* dataset's statistics. The pretrained net expects **its own** training statistics - ImageNet's mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225) - and quietly underperforms on anything else.

2026: whose shoulders you stand on

Same two recipes - but the pretrained trunk is increasingly **not** an ImageNet classifier:

- ▶ **Self-supervised backbones** learn to see from photos with **no labels at all**: shown two random crops of the same photo, the net must give them matching features - so it learns what stays the same when the view changes. Meta's **DINOv3** (2025): a 7-billion-parameter vision transformer trained this way on 1.7 billion unlabeled images, built to be used **frozen** - and distilled into laptop-sized versions, including ConvNeXt CNNs (L17).
- ▶ **Image-text backbones** (CLIP, 2021 - L17's epilogue) learned from captions, so their features already line up with words.
- ▶ Papers call Recipe 1 a **linear probe**: freeze the trunk, fit one linear layer - logistic regression, [11] - on its features. It is how these backbones are graded.

The sorting line's first try in 2026: frozen DINOv3 or CLIP features + a linear head. Fine-tune only if that is not enough. Bigger shoulders, and nobody had to label them.

Trust, but verify

The fine-tuned model scores well. Before it grades a single truckload: *what is it actually looking at?*

Grad-CAM: where is the net looking?

Three steps, no new math:

1. Take the **last conv block's feature maps** - 512 coarse 7×7 maps in ResNet-18: the most abstract "where" information left.

the photo the network sees (224 x 224)

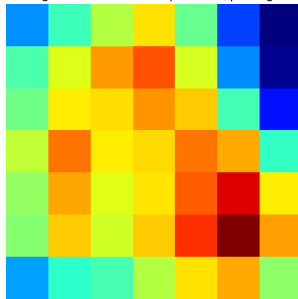


Grad-CAM: where is the net looking?

Three steps, no new math:

1. Take the **last conv block's feature maps** - 512 coarse 7×7 maps in ResNet-18: the most abstract "where" information left.
2. **Weight each map by the gradient** of the class score with respect to it - one number per map: how much the score would rise if that map fired harder.

class-weighted activation map: 7×7 ('pomegranate')



Grad-CAM: where is the net looking?

Three steps, no new math:

1. Take the **last conv block's feature maps** - 512 coarse 7×7 maps in ResNet-18: the most abstract "where" information left.
2. **Weight each map by the gradient** of the class score with respect to it - one number per map: how much the score would rise if that map fired harder.
3. **Sum, ReLU, upsample** - a heatmap of the evidence *for* that class.

One forward, one backward - works on any trained CNN, no retraining.

Selvaraju et al., "Grad-CAM", ICCV 2017.

upsampled and overlaid on the photo



The verdict on our photo

No cherry-picking: an ImageNet ResNet-18, one forward and one backward, our photo.

Top-1: **pomegranate**, $p = 0.99$. The evidence sits on the **fruit** - the aril clusters - not on the board or the basket.

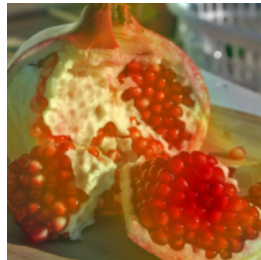
Caveat: ImageNet *has* a pomegranate class, so this is the easy case. The audit that matters is on your fine-tuned grader (HW3).

The interpretability decks ([22]-[24]) asked which **features** matter - importance, SHAP. Grad-CAM asks which **pixels**. Same question, new coordinates.

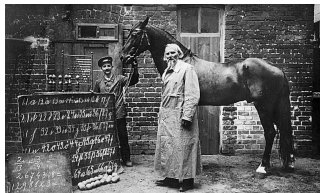
input photo



Grad-CAM for 'pomegranate' ($p = 0.99$)



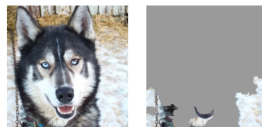
Clever Hans, the vision edition



Berlin, early 1900s: the horse **Clever Hans** "did arithmetic", tapping answers with a hoof. A 1907 investigation showed he was reading his questioner's *unconscious posture*. Right answers, wrong reason.

On the sorting line: is the model grading the **fruit**, or the **conveyor belt** behind it? High accuracy is not right reasons. Grad-CAM is the five-minute check before you trust a model.

Ribeiro et al., "Why Should I Trust You?": Explaining the Predictions of Any Classifier, KDD 2016.



(a) Husky classified as wolf (b) Explanation

Figure 11: Raw data and explanation of a bad model's prediction in the "Husky vs Wolf" task.

A wolf-vs-husky classifier, trained where every wolf photo had **snow** in the background. It looked accurate - and was a **snow detector** (Ribeiro et al. 2016 - the LIME paper, [24]).

The real budget is labels

GPUs are rented by the hour. Human attention is the expensive part.

Somebody has to draw the boxes

- ▶ Your 2,000 labeled photos meant somebody **looked at every single one** - hours of human attention, plus a second pass wherever labelers disagreed.
- ▶ Costs climb with detail: a class label is a **click**; a bounding box is a **drawing**; a per-pixel mask is a **tracing**. Each step up costs far more attention per image.
- ▶ Real projects spend their budget on **labels and label quality**, not on GPUs.

Pretraining + transfer trade **your labels** for compute **somebody else already spent**. That is why transfer is the default - economically, not just statistically.

Hold that thought: L19's detection tasks need those *boxes* - keep the price in mind.

Recap

- ▶ With 2,000 photos, **never start from scratch** - reuse the feature hierarchy someone already paid to learn.
- ▶ **Feature extraction**: freeze the trunk, train the head. **Fine-tune**: unfreeze gradually, small LR + augmentation + early stopping. The 2×2 quadrant picks between them.
- ▶ Frozen trunk $\Rightarrow$ **BN layers to eval mode**; suspect BN statistics whenever a frozen model underperforms.
- ▶ **Grad-CAM**: gradient-weighted activation maps show *where* the evidence is - the five-minute Clever Hans check.
- ▶ **Labels are the budget**. Transfer trades your labels for someone else's compute.

Next: the classifier is built and audited. But the belt camera sees *ten* fruit at once - object detection and segmentation (L19).